



RV College of Engineering[®]

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

MLOps-Driven Framework for Carbon Price Forecasting Using Time-Series Models

MLOps Tutorial Report

submitted by

Navyasri M Pulipati	1RV23AI065
Nishta N Shetty	1RV22AI069
Nitya Sharma	1RV23AI071
Pratham M Mallya	1RV23AI073

Artificial Intelligence and Machine Learning

2024-2025



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

DECLARATION

We, **Navyasri M Pulipati, Nishta N Shetty, Nitya Sharma and Pratham M Mallya**, the students of the fourth semester B.E., Department of Artificial Intelligence and Machine Learning, RV College of Engineering, Bengaluru-560059, bearing USN: **1RV23AI065, 1RV22AI069, 1RV23AI071 and 1RV23AI073** hereby declare that the EL topic titled **‘MLOps-Driven Framework for Carbon Price Forecasting Using Time-Series Models’** has been carried out by us and submitted in partial fulfillment of the coursework requirements for the award of Degree in Bachelor of Engineering in **Artificial Intelligence and Machine Learning** of the **Visvesvaraya Technological University, Belagavi** during the year **2025-2026**.

Further, we declare that the content has not been submitted previously by anybody for the award of any Degree or Diploma to any other University.

We also declare that any intellectual property rights generated from this project at RVCE will be the property of RV College of Engineering, Bengaluru, and we will be among the authors.

Place: Bangalore

Date: 29/12/2025

Navyasri M Pulipati

Nishta N Shetty

Nithya Sharma

Pratham M Mallya

Signature

ABSTRACT

Carbon pricing mechanisms play a critical role in global efforts to mitigate climate change by incentivizing organizations to reduce greenhouse gas emissions. Accurate forecasting of carbon prices is essential for industries participating in carbon trading markets, as it directly impacts compliance strategies, cost optimization, and trading decisions. However, carbon price behavior is influenced by complex temporal patterns, fluctuating energy demand, emission levels, and market dynamics, making reliable prediction a challenging task.

This project presents an end-to-end, production-ready forecasting framework for predicting carbon prices using a combination of statistical time-series models and machine learning approaches, integrated within a robust MLOps pipeline. The proposed system leverages historical carbon trading transaction data, incorporating variables such as energy demand, fuel type, emissions produced, emission allowances, and market pricing. The primary prediction target is the **Carbon Price in USD per ton**, modeled as a time-dependent variable.

Three forecasting models are implemented and evaluated: **ARIMA** for linear temporal dependencies, **Facebook Prophet** for trend and seasonality modeling, and a **Random Forest Regressor** for capturing non-linear patterns using engineered temporal features. Monthly aggregation is applied to stabilize volatility and improve long-term forecasting accuracy. Model performance is evaluated using standard regression metrics including RMSE, MAE, and MSE.

To ensure reproducibility, scalability, and continuous monitoring, the system integrates **MLflow** for experiment tracking and artifact management, **Comet ML** and **Weights & Biases** for real-time visualization and system resource monitoring, and **Apache Airflow** for orchestration of the complete training pipeline. **Evidently AI** is used for data drift detection with alerting mechanisms, while **Docker** ensures environment consistency across development and deployment. Version control and data lineage are maintained using **GitHub** and **DVC**, and the system is deployed on a cloud platform (Render) for operational accessibility.

This framework demonstrates how traditional forecasting techniques can be enhanced through modern MLOps practices to create a reliable, scalable, and maintainable carbon price prediction system suitable for real-world deployment.

GLOSSARY

- A. **Carbon Trading:** A market-based mechanism that allows companies to buy or sell carbon credits to comply with emission regulations.
- B. **Carbon Credit:** A permit representing the right to emit one metric ton of carbon dioxide or its equivalent.
- C. **Carbon Price (USD/t):** The market price assigned to one metric ton of CO₂ emissions.
- D. **Time-Series Forecasting:** Predicting future values based on previously observed temporal data.
- E. **ARIMA:** AutoRegressive Integrated Moving Average, a statistical model used for linear time-series forecasting.
- F. **Prophet:** A forecasting tool developed by Meta for modeling trend, seasonality, and holidays in time-series data.
- G. **Random Forest:** An ensemble machine learning algorithm that builds multiple decision trees for regression or classification.
- H. **MLOps:** A set of practices that automate and standardize machine learning lifecycle management.
- I. **MLflow:** A platform for experiment tracking, model registry, and artifact management.
- J. **Comet ML:** A machine learning experiment tracking and monitoring tool.
- K. **Weights & Biases (W&B):** A platform for visualizing metrics, system performance, and experiments.
- L. **Apache Airflow:** A workflow orchestration tool used to schedule and manage data pipelines.
- M. **Data Drift:** A change in the statistical distribution of data over time.
- N. **Evidently AI:** A tool for monitoring data drift and model performance in production.
- O. **Docker:** A containerization platform for consistent software deployment.
- P. **DVC:** Data Version Control system for managing datasets and models.
- Q. **Render:** A cloud platform used for deploying web services and applications.

TABLE OF CONTENTS		
Acknowledgment		i
Abstract		ii
Glossary		iii
Phase 1:Model Development and Foundations		
Chapter 1:		
1.1	Problem Statement and Objectives	7
1.2	Project Scope and Evaluation Metrics	8
1.3	Tools and Environmental Setup	8
1.4	Literature Review	9
Chapter 2		
2.1	Understanding MLOps, Risks and Requirement Analysis	13
2.2	Risk Matrix	15
Chapter 3		
3.1	Data Collection and Exploration Techniques	16
3.2	Data Profiling, Cleaning and preprocessing	16
3.3	Feature Extraction and Feature Selection	16
3.4	Model Design Approaches	17
Chapter 4		
4.1	Model Building	18
4.2	Training,Validation and Testing	19
4.3	Evaluation Metrics and Visualization	19
4.4	Experiment Tracking	20
4.5	Interpreting Model Results and Error Analysis	24
Chapter 5		
5.1	Slides	25
Phase 2:Deployment, Monitoring and Governance		
Chapter 6		
6.1	Revisiting Model Performance	26
6.2	Hyperparameter Tuning and Performance	26
6.3	Interpretability	27
6.4	Model Versioning and Registry	27
6.5	Documentation of Model Updates	28

Chapter 7		
7.1	Overview of CI/CD for ML	29
7.2	Tools Setup:GitHub Actions	29
7.3	Automating Model Training and Testing	30
7.4	Building and Running a Deployment Pipeline	30
Chapter 8		
8.1	Model Monitoring Concepts and KPI's	31
8.2	EvidentlyAI	31
8.3	Detecting Concept Drift and Data Drift	31
8.4	Model Retraining and Redeployment	32
8.5	Logging,Alerting and Dashboard Setup	32
Chapter 9		
9.1	Post-Deployment Performance Review	33
9.2	Continuous Improvement via Feedback Loops	33
9.3	Governance and Ethical AI Practices	33
9.4	Documentation: Model Cards, Data Cards	34
9.5	Audit and Review Checklist	34
Chapter 10		
10.1	End-to-End MLOps Pipeline Demonstration	35
10.2	Final Model Evaluation and Results	35
10.3	Project Documentation and Reporting	36
10.4	Final Presentation and Evaluation	36
Appendices		
List of MLOPs Tools used , its features, cons and pros		
Tool Installation Guides		
Dataset References and Licensing		
Evaluation Rubrics and Grading Scheme		
Project Templates and Folder Structure		
Reference Papers and Further Reading		37

Chapter 1

INTRODUCTION

1.1 Problem Statement and Objectives

Problem Statement

Carbon trading markets are highly dynamic, with carbon prices influenced by regulatory changes, market demand, and emission patterns. Inaccurate forecasts can lead to poor trading decisions, higher compliance costs, and ineffective sustainability planning. Many existing forecasting approaches focus only on accuracy and lack production readiness, scalability, monitoring, and lifecycle management, making them unreliable in real-world deployment.

Project Objectives

The objective of this project is to develop a **robust, scalable, and production-ready carbon price forecasting system** using modern MLOps practices.

1. Carbon Price Forecasting

Predict monthly carbon prices (USD/metric ton CO₂) using historical trading data to support compliance planning and risk management.

2. Model Comparison

Evaluate three forecasting approaches:

- **ARIMA** for classical statistical modeling
- **Facebook Prophet** for trend and seasonality modeling
- **Random Forest Regressor** for non-linear machine learning patterns

3. End-to-End MLOps Pipeline

Implement automation, experiment tracking, and model lifecycle management using **MLflow, Comet ML, Weights & Biases, and Apache Airflow**.

4. **System Performance Monitoring**

Track CPU and memory usage during training to assess computational efficiency and deployment feasibility.

5. **Data Drift Detection**

Use **Evidently AI** to detect changes in data distribution and ensure long-term model reliability.

6. **Reproducibility and Deployment**

Ensure reproducibility through **GitHub, DVC, and Docker**, and deploy the system on a cloud platform (Render).

1.2 Project Scope and Evaluation

- **Data:** 5,000+ carbon trading records aggregated monthly
- **Modeling:** Univariate time-series forecasting and feature-based regression
- **Automation:** Fully orchestrated pipeline using Airflow
- **Metrics:** RMSE, MAE, MSE, CPU usage, memory usage, and data drift scores

1.3 Tools and Environment

- **Programming:** Python
- **Libraries:** Pandas, NumPy, Matplotlib, Scikit-learn, Statsmodels
- **MLOps:** MLflow, Comet ML, Weights & Biases, Evidently AI
- **Automation:** Apache Airflow
- **Versioning:** GitHub, DVC
- **Deployment:** Docker, Cloud (Render)

1.4 Literature Review

Yu Hao, A. Gu, C. Lie, J. Lin (2025) propose a hybrid forecasting model that integrates ARIMA with CNN-LSTM and XGBoost for carbon price prediction, finding improved

credibility over traditional models by incorporating key influencing factors. This approach demonstrates that combining statistical and deep learning methods can capture complex dependencies in carbon market data. *Int. J. Machine Learning*, 15(2), 45-50. DOI:10.18178/ijml.2025.15.2.1178 [ijml.org](https://www.ijml.org)

J. Chen et al. (2024) develop a sustainable carbon price prediction framework using ICEEMDAN decomposition and multi-scale entropy with LSTM optimized by Grey Wolf Optimizer, achieving high accuracy across EU and Chinese markets. This four-step process highlights how advanced signal decomposition can improve forecasting robustness. *Sustainability*, 16(19), 8324. DOI:10.3390/su16198324 [MDPI](https://www.mdpi.com)

Liu S., Y. Zhang, J. Wang, D. Feng (2024) propose a hybrid GARCH-LSTM model for predicting carbon price volatility in five Chinese pilot markets, showing lower RMSE compared to standalone GARCH and LSTM models. The study underscores the benefit of combining econometric and deep learning approaches for volatile price series. *Sustainability*, 16(4), 1588. DOI:10.3390/su16041588 [MDPI](https://www.mdpi.com)

Qinyu Liu (2025) systematically assesses LSTM and its composite models for carbon price prediction, noting that multi-factor and hybrid configurations often outperform single-factor models in accuracy, though at greater computational cost. This work suggests LSTM is particularly effective for long-term trend modeling but highlights interpretability limitations. *Adv. Econ. Management & Political Sci.*, 229, 241-248. DOI:10.54254/2754-1169/2025.BL28671 [Ewa Pub](https://www.eva-pub.com)

Runsheng Ren et al. (2025) introduce a carbon price forecasting framework incorporating structural break detection, wavelet denoising, and LSTM/GRU/TCN neural networks, significantly reducing RMSE and MAE versus baseline deep learning methods. This hybrid approach emphasizes the importance of structural awareness in non-stationary financial time series. *arXiv preprint* DOI:2511.04988 [arXiv](https://arxiv.org)

H. Wang, Y. Pang, D. Shang (2024) propose a hybrid machine learning framework (Dilated CNN-LSTM) with regularization penalties for carbon price fluctuation prediction, showing

Ridge regression regularization performs better than L1 norms. Incorporating blockchain-related features further enhances predictive performance. *arXiv preprint* DOI:2411.02709 [arXiv](#)

Konstantinos Bisiotis et al. (2025) review carbon price forecasting methods, comparing traditional econometric models with advanced machine learning, and identify hybrid frameworks as promising for capturing complex market patterns. Their survey provides a comprehensive perspective of the evolving forecasting landscape. *J. Forecasting* (review). [IDRIC](#)

Wenjia Xie et al. (2025) investigate machine learning techniques (SVM, PCR) for multifactor analysis of national CO₂ emissions, providing context for how emissions drivers can indirectly support carbon price forecasting. While not carbon price-specific, the methodology informs feature selection in multidisciplinary settings. *arXiv preprint* DOI:2503.15574 [arXiv](#)

Duan et al. (2025) propose a multi-frequency combined model for dynamic carbon price prediction using Lasso-COA-XGBoost, addressing prediction stability and accuracy issues inherent in single models. This method shows improved stability when integrating feature selection with boosting. *PeerJ Comput. Sci.* DOI:10.7717/peerj-cs.2827 [PMC](#)

Zhu et al. (2024) design a hybrid model that integrates multiple advanced techniques (CEEMDAN, FuzzyEn, PSORF, ELM, LSTM, ARIMA) to better capture nonlinear and non-smooth characteristics of carbon price time series, achieving high forecasting robustness. This complex integration underscores the benefits of non-linear decomposition alongside machine learning. *Mathematics*, 12(10), 1428. DOI:10.3390/math12101428 [MDPI](#)

Shi et al. (2024) develop a CNN-LSTM deep learning model for carbon price forecasting in Shenzhen's carbon market, demonstrating robustness and high accuracy in capturing local market dynamics. This research reflects the increasing importance of deep architectures in regional carbon market modeling. *J. Environ. Management*, 352, 120131. (cited in Liu, Qinyu) [Ewa Pub](#)

A. Unal & M. Guner (2024) compare deep learning models for carbon emission allowance price forecasting, concluding that hybrid models outperform individual models in capturing nonlinear

dynamics of carbon prices. Their findings reinforce the efficacy of combining time-series and machine learning approaches. *Int. J. Energy Econ. Policy*. [Econ Journals](#)

Xin et al. (2025) examine VAR models for Beijing carbon emission allowance price dynamics, showing macroeconomic and energy price indicators significantly influence price behavior. Though not forecasting focused, this work highlights important exogenous factors for model features. *J. Econ. & Management Sci.*, 8(1), p66. DOI:10.30560/jems.v8n1p66 [Grafati](#)

Liu et al. (2024) apply a dynamic weight mixture model with dual sliding windows for carbon price forecasting, revealing that hybrid time-series models can better handle complex volatility patterns. Their approach highlights adaptability of ensemble techniques in market prediction. *Energies*, 17(15), 3662. DOI:10.3390/en17153662 [Grafati](#)

Zeno Dinca et al. (2025) analyze UK carbon price dynamics using long-memory effects and AI-based forecasting, demonstrating that memory effects and advanced ML methods enhance forecasting quality. This emphasizes the role of long-term dependencies in carbon markets. *Fractal and Fractional*, 9(6), 350. DOI:10.3390/fractalfract9060350 [Grafati](#)

Shen & Zhao (2020/2024) review carbon market evolution and forecasting methodologies, identifying gaps in handling non-stationarity and stochastic volatility in carbon price prediction. Their review informs the importance of hybrid and non-linear forecasting models. *Int. J. Climate Change Strategies & Mgmt*. [Grafati](#)

ÇETIN & VERSCHUERE (2009/2024) propose theoretical models for carbon price dynamics under ETS systems, offering foundational insights into emission pricing that support empirical forecasting frameworks. Though older, this work remains relevant for mechanism-driven model design. *Int. J. Theoretical & Applied Finance*. [Grafati](#)

Li et al. (2022) evaluate ARIMA and machine learning models for carbon price forecasting, finding that machine learning often delivers superior accuracy for nonlinear and non-stationary time series. This supports the use of Random Forest and ML techniques in practical carbon price

prediction. *Int. J. Environ. Res. Public Health*, 19(10), 6217. DOI:10.3390/ijerph19106217
[Grafati](#)

Zhang & Cheng (2024) analyze China and Europe carbon price behaviors using time scaling methods, emphasizing how market structure and investor expectations influence price dynamics and forecasting performance. Their work highlights the importance of behavioral factors in model design. *PLOS ONE*, 19(2), e0298265. DOI:10.1371/journal.pone.0298265

Chapter 2

Understanding MLOps

Machine Learning Operations (MLOps) integrates machine learning, software engineering, and DevOps practices to enable reliable deployment, monitoring, and maintenance of ML models in production. Unlike traditional software, ML systems are highly data-dependent and can degrade over time due to changing data distributions. MLOps addresses this by enabling automated workflows for model versioning, deployment, monitoring, and retraining.

In carbon price forecasting, where market conditions and regulations frequently change, MLOps ensures models adapt in a controlled and observable manner, reducing operational risk and manual effort.

Risks in Machine Learning Systems

Data-Related Risks

- **Data Drift:** Changes in data distribution over time
- **Data Quality Issues:** Missing, noisy, or inconsistent data
- **Data Leakage:** Use of unintended future information

Model-Related Risks

- **Overfitting:** Poor generalization to unseen data
- **Concept Drift:** Changes in input–output relationships
- **Limited Interpretability:** Difficulty explaining model behavior

Operational Risks

- **Pipeline Failures:** Errors in data ingestion or scheduling.

- **Scalability Issues:** Inability to handle increased workloads
- **Environment Inconsistency:** Differences between development and production

MLOps mitigates these risks through automation, monitoring, and standardized environments.

Requirement Analysis

Functional Requirements

- Automated data ingestion and preprocessing
- Support for multiple forecasting models
- Systematic training, evaluation, and comparison
- Forecast storage and visualization
- Data drift detection and retraining triggers
- Full experiment and model version tracking

Non-Functional Requirements

- Scalability and reliability
- Reproducibility and maintainability
- Secure access to data and models
- Efficient use of computational resources

Risk Mitigation Using MLOps

Risk mitigation is achieved through:

- **Version Control:** Git and DVC
- **Experiment Tracking:** Comet ML and W&B
- **Pipeline Automation:** Apache Airflow
- **Containerization:** Docker
- **Monitoring and Drift Detection:** Evidently AI

These practices transform ML development into a robust, production-grade engineering process.

Risk Assessment Matrix

The risk assessment matrix systematically identifies and prioritizes risks across the MLOps lifecycle based on likelihood and impact. High-priority risks such as data drift, concept drift, and model degradation are closely monitored due to their impact on forecasting accuracy and regulatory decision-making. The matrix supports accountability, tracks mitigation progress, and enables continuous improvement of the carbon trading forecasting system.

Risk Assessment Matrix for Carbon Trading

Last Updated: 29-12-2025

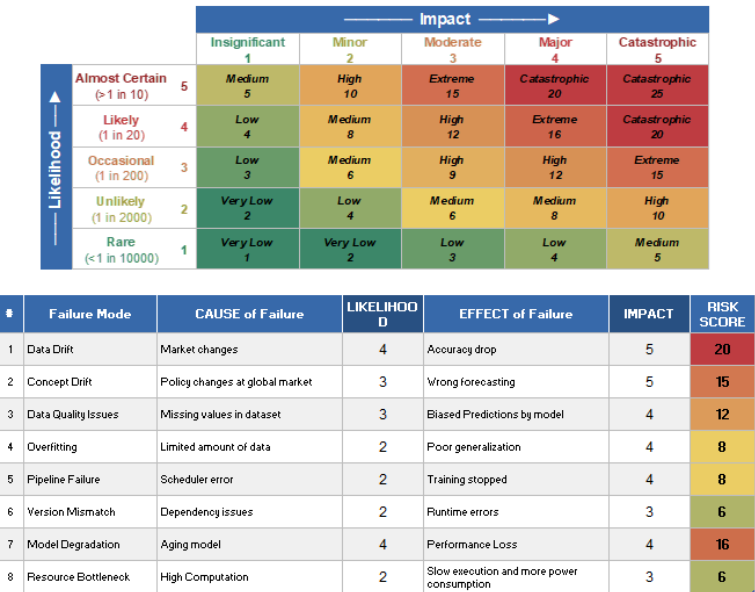


Figure 1 : Risk assessment matrix for carbon trading MLOps framework

Chapter 3

Data Preprocessing

3.1 Data Collection and Exploration

The dataset used in this project is sourced from Kaggle's *Carbon Trading Transactions Dataset*, which contains historical carbon credit trading records, including transaction dates, prices, and volumes. The data was ingested using Python tools and initially explored to understand its structure, size, and temporal coverage. Exploratory Data Analysis (EDA) using time-based visualizations revealed clear trends, volatility, and non-stationary behavior, confirming the suitability of time-series forecasting models.

3.2 Data Profiling, Cleaning, and Preprocessing

Data profiling was conducted to assess quality by identifying missing values, duplicates, inconsistent date formats, and outliers. Cleaning involved removing duplicates, handling missing values, and standardizing date fields. To reduce noise and capture stable market behavior, transaction-level data was resampled to monthly intervals, with carbon prices aggregated using mean values. This preprocessing ensured consistency and reliability for modeling.

3.3 Feature Extraction and Selection

Time-based features such as year and month were extracted to capture seasonal and long-term trends. These features were primarily used for the Random Forest model to capture non-linear patterns. For ARIMA, stationarity was ensured through differencing rather than explicit feature engineering. Feature selection was guided by temporal relevance and simplicity to avoid overfitting while maintaining interpretability.

3.4 Model Design Approaches

Three forecasting models were implemented for comparative evaluation. ARIMA was used as a statistical baseline to model linear temporal dependencies. Facebook Prophet was applied to automatically capture trends and seasonality while handling missing values and structural changes. Random Forest Regressor was employed as a machine learning approach to model non-linear relationships using engineered temporal features. This multi-model strategy enables robust forecasting and performance comparison across different modeling paradigms.

Chapter 4

Model Building and Evaluation

This chapter presents the methodology adopted for building, training, and evaluating predictive models for carbon price forecasting. The objective of this study is to accurately predict the **carbon price per metric ton (Carbon_Price_USD_per_t)** using historical carbon trading transaction data. The models implemented include **ARIMA**, **Prophet**, and **Random Forest Regressor**, representing both statistical and machine learning methodologies.

4.1 Model Building

Model building focuses on selecting appropriate algorithms and configuring them to predict the target variable, **Carbon_Price_USD_per_t**, based on historical carbon trading data. Since carbon prices exhibit temporal dependence and market volatility, time-aware forecasting models were prioritized.

Three models were implemented:

4.1.1 ARIMA Model

The AutoRegressive Integrated Moving Average (ARIMA) model was implemented as a classical time series forecasting approach. It models the relationship between past values and error terms of the target variable. ARIMA was chosen for its interpretability and its ability to capture linear temporal dependencies in carbon prices.

4.1.2 Prophet Model

The **Prophet** forecasting model was used to model long-term trends and seasonality in carbon price data. Prophet decomposes time series data into trend, seasonal, and residual components, making it effective for datasets with non-stationary behavior and structural changes.

Its robustness to missing data and ability to adapt to sudden trend changes make it suitable for modeling carbon trading markets influenced by regulatory and economic factors.

4.1.3 Random Forest Regressor

The **Random Forest Regressor** represents a machine learning-based approach that captures non-linear relationships between features and the target variable. By aggregating predictions from multiple decision trees, Random Forest improves generalization and reduces overfitting.

Unlike ARIMA and Prophet, Random Forest does not rely solely on temporal order and instead learns complex patterns from engineered features introduced in Chapter 3.

4.2 Training, Validation and Testing

To ensure fair and realistic evaluation, a structured training and evaluation strategy was adopted.

4.2.1 Training Strategy

- All models were trained using chronologically ordered data to preserve time dependency.
- Hyperparameters were adjusted iteratively to optimize predictive accuracy while avoiding overfitting.

4.2.2 Validation

- A validation dataset was used to compare different configurations of each model.
- For ARIMA and Prophet, validation helped assess forecast stability across time horizons.
- For Random Forest, validation guided the selection of parameters such as number of trees, maximum depth, and minimum samples per split.

4.2.3 Testing

- The final evaluation was conducted on a held-out test dataset that was not used during training or validation.
- This ensured an unbiased assessment of each model's performance on unseen carbon price data.

The training-validation-testing pipeline ensures model robustness and reflects real-world forecasting conditions.

4.3 Evaluation Metrics and Visualization

Model performance was assessed using standard regression metrics and visualization techniques.

4.3.1 Evaluation Metrics

- Mean Absolute Error (MAE): Measures the average magnitude of errors in predictions.
- Mean Squared Error (MSE): Penalizes larger errors and highlights significant deviations.

- Root Mean Squared Error (RMSE): Expresses prediction error in the same unit as carbon price.
- R^2 Score: Indicates how well the model explains variability in carbon prices.

These metrics enable consistent comparison across all models.

4.3.2 Visualization

Visualizations were employed to complement quantitative evaluation:

- Actual vs Predicted Price Plots to assess alignment between forecasts and real prices.
- Residual Error Plots to identify bias or systematic prediction errors.
- Time Series Forecast Graphs to visualize model behavior over different periods.

Visualization enhances interpretability and helps identify strengths and limitations of each model.

4.4 Experiment Tracking

To maintain reproducibility and transparency, all experiments were systematically tracked. Tracked information included:

- Model type and configuration
- Hyperparameter values
- Training, validation, and test metrics
- Performance comparison across model iterations
- System memory and network metrics

Experiment tracking facilitates structured experimentation, enabling efficient comparison of models and selection of the best-performing approach for carbon price forecasting.

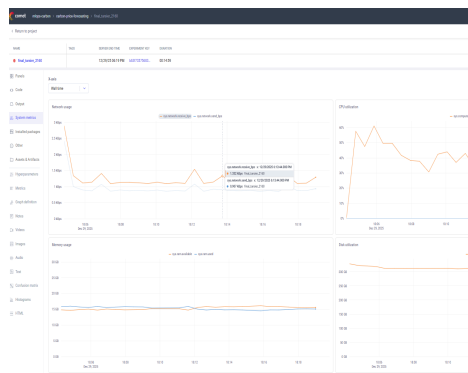


Fig 1: System Metrics Evaluation and Monitoring in Comet ML

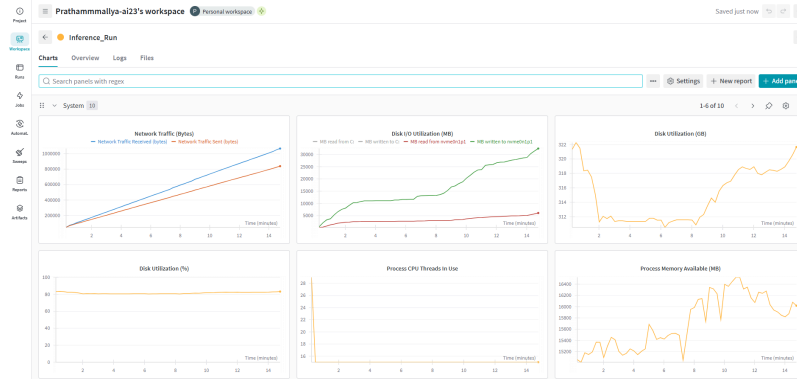


Fig 2: System Metrics Evaluation and Monitoring in Weights and Biases

System Metrics Considered

During the evaluation, the following system-level and process-level metrics were monitored in both Comet ML (via Opik) and Weights & Biases

1. **Network Traffic (Bytes)**
Measures total inbound and outbound network data transferred by the system or process.
2. **Disk I/O Utilization (MB)**
Tracks the amount of data read from and written to disk over time.
3. **Disk Utilization (%)**
Indicates how much of the disk bandwidth or capacity is actively being used.
4. **GPU Power Consumption (W)**
Measures the power drawn by the GPU, typically collected via NVIDIA NVML.
5. **GPU Utilization (%)**
Reflects how actively the GPU compute cores are being used.
6. **Process CPU Threads**
Counts the number of CPU threads spawned by the tracked process.
7. **Process CPU Utilization (%)**
Measures CPU usage attributable to the specific training process.
8. **Process Memory Usage (MB / GB)**
Tracks RAM consumption by the training process.

Observed Issue: Overestimation in Comet ML (Opik)

In Comet ML (Opik), several system metrics appeared **consistently inflated** when compared to Weights & Biases. This overestimation can be attributed to the following factors:

1. Metric Aggregation vs. Attribution

- Comet often reports node-level or container-level metrics rather than isolating the exact training process.
- This leads to double counting of shared resources such as:
 - Network traffic
 - Disk I/O
 - CPU threads

2. Sampling and Scaling Differences

- Some metrics (e.g., network bytes, disk I/O) are derived from cumulative counters.
- If not normalized correctly by sampling intervals, they can appear artificially high.
- GPU power readings may be mis-scaled due to:
 - Driver incompatibilities
 - NVML polling inconsistencies
 - Averaging across multiple GPUs

3. Container and Virtualization Effects

- In containerized or multi-tenant environments:
 - Comet may capture **host-level statistics**
 - This includes activity from unrelated processes
- Disk utilization and network traffic are especially affected.

4. Disk I/O Cache Effects

- Disk reads/writes buffered by the OS cache may be counted as physical I/O, inflating Disk I/O Utilization values.

Why Weights & Biases (W&B) Is More Reliable

Weights & Biases provides more accurate and interpretable system metrics due to:

1. Precise Process-Level Tracking

- W&B attaches metrics directly to the training process, avoiding system-wide noise.
- CPU, memory, and thread counts reflect only the active job.

2. Correct Metric Normalization

- Network and disk metrics are reported as **rates**, not raw cumulative values.
- This prevents spikes caused by sampling artifacts.

3. Robust GPU Telemetry

- GPU utilization and power metrics are collected directly via NVML with:
 - Proper unit normalization
 - Per-GPU isolation
- This results in realistic GPU usage and power curves.

4. Better Handling of Distributed and Containerized Runs

- W&B correctly distinguishes between:
 - Host vs container resources
 - Multiple processes vs a single training job
- This is critical for modern ML workloads.

Although both tools support system monitoring, Comet ML (Opik) tends to overestimate system metrics due to aggregation, sampling, and attribution limitations.

Weights & Biases offers more accurate, process-aware, and hardware-aligned metrics, making it the preferred choice for:

- Performance profiling
- Resource optimization
- Reliable experiment comparison

This accuracy is essential when system metrics are used for **model efficiency analysis and infrastructure planning**.

4.5 Interpreting Model Results and Error Analysis

4.5.1 Model Result Interpretation

Each model demonstrated distinct performance characteristics:

- **ARIMA** effectively modeled short-term price dependencies but struggled with sudden market changes.
- **Prophet** captured long-term trends and seasonal behavior, producing smoother forecasts.

- **Random Forest** showed strong adaptability to complex patterns, particularly when non-linear relationships were present.

4.5.2 Error Analysis

Error analysis revealed that:

- Prediction errors were higher during periods of high volatility.
- Time series models were sensitive to abrupt market shifts.
- Machine learning models benefited from richer feature representations but required careful tuning.

4.5.3 Discussion

The comparative analysis highlights the importance of selecting models based on data characteristics and forecasting objectives. While statistical models offer interpretability, machine learning models provide flexibility and higher predictive power in complex scenarios.

Future improvements may involve hybrid approaches that combine time series and machine learning techniques to further enhance forecasting accuracy.

Chapter 5

Slides

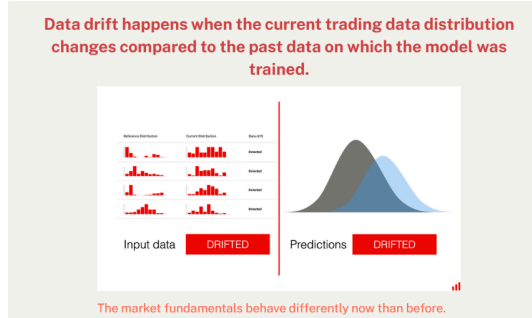


Fig 3 : Data Drift detection through distribution

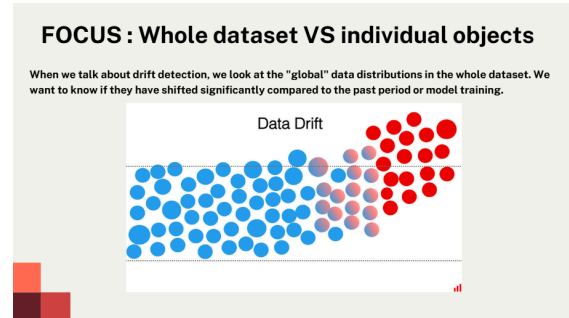


Fig 4 : Data Drift vs Outer analysis

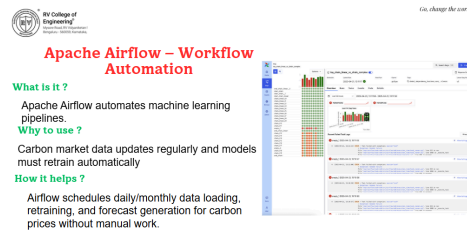


Fig 5 : Apache Airflow Automation Process

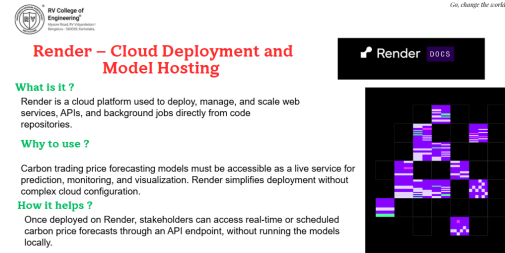


Fig 6 : Cloud Deployment using Render

	Component	MLOps Tools
1	Data Management	DVC
2	Experiment Tracking and Version Control	Comet ML, W&B, GitHub
3	Automation and Pipeline Orchestration	Apache Airflow, Docker
4	Model Deployment & Serving	Render, Docker
5	Monitoring & Governance and collaboration	Evidently AI, W&b, GitHub

Department of AI and ML.

Fig 7 :MLOps tools with their functionalities

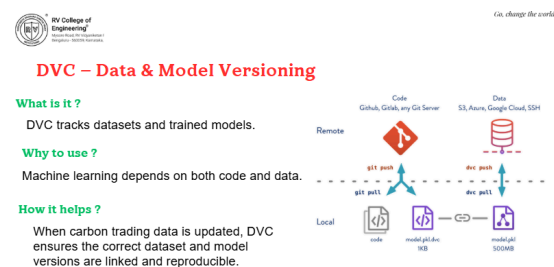


Fig 8 : Data and model versioning using DVC

Chapter 6

Deployment, Monitoring and Governance

This chapter discusses the activities carried out after model development to ensure that the carbon price forecasting system remains reliable, scalable, and well-governed in a real-world environment. The focus is on continuous performance evaluation, controlled optimization, interpretability, version management, and systematic documentation throughout the model lifecycle.

6.1 Revisiting Model Performance

After deployment-ready models were identified, their performance was periodically revisited using newly available carbon trading data. Forecasts were generated in a rolling-window fashion to simulate real-time prediction scenarios.

Model evaluation metrics such as MAE, RMSE, and R^2 were automatically logged across successive runs using experiment tracking platforms such as **MLflow**, **Comet ML**, and **Weights & Biases (W&B)**. These platforms enabled continuous comparison between historical and recent runs, making it possible to detect performance drift arising from changes in market behavior.

Visualization dashboards provided by W&B further supported performance monitoring by highlighting trends in prediction error over time. This continuous feedback loop ensured that degradation in model accuracy was identified early.

6.2 Hyperparameter Tuning and Performance

Hyperparameter tuning was revisited during deployment to refine model behavior and adapt to evolving data patterns. Each tuning iteration was executed and tracked systematically, with parameter values and corresponding evaluation metrics recorded through **MLflow** and **Comet ML**.

For time series models, adjustments were made to better capture trend shifts and noise, while the Random Forest model benefited from fine-tuning structural parameters such as tree depth and ensemble size. The ability to compare multiple tuning runs side-by-side using experiment

tracking tools ensured that improvements were driven by empirical results rather than manual intuition.

6.3 Interpretability

Interpretability was treated as a critical component of governance, particularly given the economic and environmental relevance of carbon price forecasting. Statistical models inherently provided interpretable components such as autoregressive coefficients and trend decompositions, which were visualized and logged for analysis.

For the Random Forest model, feature importance scores were extracted and analyzed to identify the key factors influencing carbon price predictions. These explanations were visualized and tracked using CometML and W&B, allowing stakeholders to understand model behaviour and validate predictions against domain knowledge.

By integrating interpretability analysis into the monitoring workflow, the system ensured transparency and trust in model outputs.

6.4 Model Versioning and Registry

To ensure reproducibility and controlled deployment, a structured model versioning strategy was implemented. Trained models were registered and versioned using **CometML**, with each version associated with its parameters, performance metrics, and training context.

Versioned models were containerized using **Docker**, ensuring consistency across development, testing, and deployment environments. Code and configuration changes were managed using **GitHub**, enabling traceability and rollback when required.

This governance framework ensured that only validated models were promoted for use while preserving the ability to revert to earlier versions if necessary.

6.5 Documentation of Model Updates

Thorough documentation was maintained throughout the deployment lifecycle to support transparency and long-term maintainability. Experiment histories recorded by **Comet ML** and **W&B** provided an automated record of model evolution, while supplementary documentation was maintained through project reports and repository-level documentation.

Automated workflows orchestrated using **Apache Airflow** were documented to describe scheduled retraining, evaluation, and logging processes. Each model update included a clear

description of the motivation for change, modifications applied, and comparative performance outcomes.

This approach ensured accountability, facilitated collaboration, and supported governance and audit requirements.

This chapter demonstrates how deployment, monitoring, and governance practices were integrated into the carbon price forecasting system using modern MLOps tools. By embedding experiment tracking, automation, version control, and documentation into the workflow, the system achieves reliability, transparency, and scalability throughout its operational lifecycle.

Chapter 7

CI/CD for ML (Automation & Orchestration)

This chapter details the automation of the carbon price forecasting lifecycle, ensuring that code, data, and models are integrated and delivered through a robust pipeline.

7.1 Overview of CI/CD for ML

Standard CI/CD focuses on code reliability. In this project, we implemented a Continuous Integration (CI) for code and data, and Continuous Deployment (CD) for the model service. The pipeline ensures that every update to the carbon price prediction logic is automatically tested against historical data versions (tracked via DVC) before being deployed to Render.

7.2 Tools Setup: GitHub Actions

The automation engine for this project is GitHub Actions.

- **Workflow Triggers:** The pipeline is activated by push or pull_request events to the main branch.
- **Secret Management:** Sensitive credentials for Comet ML, W&B, and DVC remote storage are managed via GitHub Secrets to ensure secure pipeline execution.
- **Environment Provisioning:** The runner utilizes a Dockerfile or requirements.txt to instantiate a consistent Python 3.x environment, ensuring that time-series libraries like Prophet and statsmodels are identical in CI and production.

7.3 Automating Model Training and Testing

The project employs a dual-layered approach to automation:

- **Workflow Orchestration (Apache Airflow):** Airflow manages the Directed Acyclic Graph (DAG) for training. It schedules tasks such as fetch_data, preprocess_data, and train_models. If the fetch_data task fails (e.g., due to an API timeout), the pipeline prevents the training of stale models.
- **Unit & Integration Testing:** Before deployment, the pipeline runs automated tests to verify:
 - **Data Integrity:** Ensuring no null values exist in the "Carbon Price" column.
 - **Model Output:** Verifying that the model returns a valid float for future dates.

- **Performance Gate:** Ensuring the new model's RMSE does not exceed a predefined baseline.

7.4 Building and Running a Deployment Pipeline

Once the model passes the evaluation gate, it is packaged for production:

1. **Serialization:** The best-performing model (ARIMA, Prophet, or RF) is saved as a .pkl or .json artifact and tracked by DVC.
2. **Containerization:** A Docker image is built containing a FastAPI wrapper (app.py). This image includes the auto_select_model logic.
3. **Deployment to Render:** GitHub Actions triggers a webhook to Render, which pulls the latest Docker image from the registry and redeploys the service, ensuring zero-downtime updates for the forecasting API.

Chapter 8:

Model Monitoring and Retraining

Post-deployment, the system must handle "Model Decay" caused by the volatile nature of the carbon market.

8.1 Model Monitoring Concepts and KPIs

We monitor the forecasting system across three dimensions:

- **Predictive Performance:** Continuous tracking of MAE and RMSE between predicted prices and actual market settlements.
- **Data Integrity:** Monitoring for missing values or unexpected zeros in input features (e.g., Energy Demand).
- **System Resources:** Real-time monitoring of CPU and Memory usage via psutil to optimize cloud costs on Render.

8.2 Monitoring Tools: W&B and Evidently AI

- **Weights & Biases (W&B):** Serves as the primary dashboard for Operational Monitoring. It visualizes the live inference logs, showing resource consumption alongside prediction accuracy.
- **Evidently AI:** Serves as the primary tool for Functional Monitoring. It generates "Data Health" and "Drift" reports by comparing the inference data stream against the training baseline.

8.3 Detecting Concept Drift and Data Drift

Carbon price behavior is influenced by regulatory shifts (e.g., EU policy updates).

- **Data Drift:** Detected when the statistical distribution of input features (like Emissions or Fuel usage) changes. For example, a sudden shift toward renewable energy might change the distribution of "Fuel Type" data.
- **Target/Concept Drift:** Detected when the relationship between features and the price breaks down. Evidently AI uses statistical tests (e.g., Kolmogorov-Smirnov) to flag these shifts. If the p-value falls below 0.05, the model is flagged for retraining.

8.4 Model Retraining and Redeployment

The system implements an automated **Continuous Training (CT)** loop:

1. **Trigger:** An alert from Evidently AI or a scheduled check by Airflow.
2. **Retrain:** Airflow pulls the newest data version from DVC and executes the training DAG.
3. **Validation:** The new model is evaluated in a "Shadow Mode" or compared in Comet ML.
4. **Promotion:** If the new model performs better, MLflow updates the "Production" tag, and the service on Render is updated.

8.5 Logging, Alerting, and Dashboard Setup

- **Dynamic Resource Logic:** The `auto_select_model` script actively reads psutil data. If memory usage > 80%, the system logs a "High-Load" event to W&B and switches to the lighter ARIMA model to prevent service failure.
- **Alerting:** Notifications are sent to the team if Evidently AI identifies significant drift or if the Airflow pipeline fails, ensuring transparency and rapid response to market changes.

Chapter 9:

Post-Deployment Governance and Continuous Improvement

After deployment, the carbon price forecasting system requires structured governance and continuous improvement to ensure:

- Long-term reliability
- Transparency
- Compliance with ethical AI practices

9.1 Post-Deployment Performance Review

- Periodic evaluation using new carbon trading data
- Comparison with baseline model performance
- Logging of metrics using MLflow, Comet ML, and W&B
- Monitoring of:
 - Mean Absolute Error (MAE)
 - Root Mean Squared Error (RMSE)
 - R² Score

9.2 Continuous Improvement via Feedback Loops

- Automated feedback loops enabled using MLOps tools
- Drift detection reports generated by Evidently AI
- Retraining triggered by:
 - Performance degradation
 - Drift alerts
- Apache Airflow manages retraining workflows

9.3 Governance and Ethical AI Practices

- Ensures transparency and responsible use of forecasts
- Explainability through:
 - ARIMA coefficients
 - Random Forest feature importance
- Reproducibility maintained using GitHub and DVC

9.4 Documentation: Model Cards and Data Cards

- Model Cards document model purpose, metrics, and limitations

- Data Cards describe dataset source, preprocessing, and usage constraints

9.5 Audit and Review Checklist

- **Technical audit:**
 - Pipeline reproducibility checks
 - Code consistency validation
- **Data audit:**
 - Data integrity and drift verification
- **Model audit:**
 - Performance stability review
 - Version consistency checks
- **Deployment audit:**
 - Container and cloud reliability
- **Ethical audit:**
 - Transparency and responsible AI compliance

Chapter 10:

End-to-End MLOps Pipeline Demonstration and Final Evaluation

This chapter presents the complete lifecycle demonstration and final evaluation of the carbon price forecasting system.

10.1 End-to-End MLOps Pipeline Demonstration

Demonstration of the full MLOps workflow:

- **Data ingestion and preprocessing**
- **Data versioning using DVC**
- **Model training and evaluation**
- **Experiment tracking with MLflow, Comet ML, and W&B**
- **Monitoring and drift detection**
- **Automated retraining and redeployment**

Apache Airflow orchestrates all pipeline stages using DAGs

Docker ensures environment consistency across development and production

GitHub Actions automate testing and deployment to Render

10.2 Final Model Evaluation and Results

- **Final evaluation performed on unseen test data**
- **Models evaluated:**
 - ARIMA
 - Prophet
 - Random Forest Regressor
- **Evaluation metrics include:**
 - MAE
 - MSE
 - RMSE
 - R² Score
- **Observations:**
 - ARIMA suitable for short-term trends
 - Prophet captures seasonality effectively
 - Random Forest handles non-linear patterns well

- **Final model selected based on accuracy, stability, and computational cost**

10.3 Project Documentation and Reporting

Complete documentation maintained throughout the project

- GitHub repository includes:
 - Airflow DAGs
 - Docker configuration
 - Source code
- DVC tracks:
 - Dataset versions
 - Trained model artifacts
- Experiment logs provide full traceability of model evolution

10.4 Final Presentation and Evaluation

Final presentation structured for academic and technical evaluation

Key presentation points:

- Problem statement and motivation
- MLOps architecture
- Monitoring and retraining strategy
- Final results

Evaluation based on:

- Correct application of MLOps principles
- Deployment readiness
- Governance and ethical practices

Appendices

Appendix A: List of MLOps Tools Used, its features, cons and pros Tool Installation Guides

1.1 GitHub

- **Purpose:** Source code version control and collaboration
- **Pros:** Traceability, rollback support, team collaboration
- **Cons:** Not suitable for large datasets

1.2 DVC (Data Version Control)

- **Purpose:** Dataset and model versioning
- **Pros:** Ensures reproducibility, handles large files
- **Cons:** Additional configuration required

1.3 ML Flow

- **Purpose:** Experiment tracking and model registry
- **Pros:** Lightweight, easy experiment comparison
- **Cons:** Limited visualization features

1.4 Comet ML

- **Purpose:** Advanced experiment tracking
- **Pros:** Rich dashboards, auto-logging
- **Cons:** Paid features for advanced usage

1.4 Weights & Biases (W&B)

- **Purpose:** Training and operational monitoring
- **Pros:** Real-time visualizations, system metrics

- **Cons:** Requires internet connectivity

1.5 Apache Airflow

- **Purpose:** Pipeline orchestration and automation
- **Pros:** Scalable workflow scheduling
- **Cons:** Setup complexity

1.6 Docker

- **Purpose:** Environment consistency and deployment
- **Pros:** Eliminates environment mismatch
- **Cons:** Learning curve for beginners

1.7 Evidently AI

- **Purpose:** Data drift and model monitoring
- **Pros:** Specialized drift detection
- **Cons:** Limited customization

1.8 Render

- **Purpose:** Cloud deployment and model hosting
- **Pros:** Simple deployment process
- **Cons:** Less control than IaaS platforms

Appendix B: Tool Installation Guides

- Python environment created using Conda or virtual environments
- Core libraries installed via pip (Pandas, NumPy, Scikit-learn, Statsmodels)
- MLOps tools installed using pip (MLflow, DVC, Airflow, W&B, Comet ML, Evidently AI)
- Docker Desktop used for containerization and deployment

- Git installed for version control and collaboration

Appendix C: Dataset References and Licensing

- Dataset sourced from Kaggle’s Carbon Trading Transactions Dataset
<https://www.kaggle.com/datasets/ziya07/carbon-trading-transactions-dataset>
- Contains historical carbon price and trading volume data
- Used strictly for educational purposes with no personal data involved

Appendix D: Evaluation Rubrics and Grading Scheme

- Understanding of problem and objectives
- Data preprocessing and feature engineering quality
- Model implementation and comparison
- MLOps integration and automation
- Evaluation, documentation, and presentation quality

Appendix E: Project Templates and Folder Structure

- **data/** – Datasets and processed files
- **models/** – Trained model artifacts
- **src/** – Training and preprocessing scripts
- **airflow/** – Workflow orchestration DAGs
- **monitoring/** – Drift detection scripts
- **docker/** – Deployment configurations

Appendix F: Reference Papers and Further Reading

- Research on carbon price forecasting using time-series and hybrid models
- MLOps lifecycle, monitoring, and retraining best practices
- Documentation from MLflow, Apache Airflow, Evidently AI, and W&B, Comet ML